

多版本机制

数据库管理系统（DBMS）会为数据库中的单个逻辑对象维护多个物理版本：

- 当事务对对象进行写入操作时，数据库管理系统会为该对象创建新版本。
- 当事务读取对象时，会读取该对象在事务启动时存在的最新版本。
- 读取操作无需等待，系统会立即返回合适的版本。

多版本方案通过保留数据项的旧版本以提高并发性。主要变体包括：

- 多版本时间戳排序 (Multiversion Timestamp Ordering)
- 多版本两阶段锁定 (Multiversion 2-Phase Locking)
- 快照隔离 (Snapshot Isolation)

主要观点：

- 每次成功提交事务时，都会创建一个新版本。
- 使用时间戳/序列号来标识版本。
- 当发出读取请求时，系统会返回在事务开始时存在的最新版本。

多版本时间戳排序

每个数据项 Q 具有一个版本序列 $\langle Q_1, Q_2, \dots, Q_m \rangle$ 。每个版本 Q_k 包含三个数据字段：

- **Content**： Q_k 版本的值。
- **W-timestamp(Q_k)**：创建（写入）版本 Q_k 的事务时间戳。
- **R-timestamp(Q_k)**：成功读取 Q_k 版本的事务中时间戳的最大值。

假设事务 T_i 执行 **read(Q)** 或 **write(Q)** 操作。令 Q_k 表示 Q 的版本，其 **W-timestamp** 为小于或等于 $TS(T_i)$ 的最大值。

1. 若事务 T_i 发出 **read(Q)** 请求

- 返回版本 Q_k 的内容。
- 若 $R\text{-timestamp}(Q_k) < TS(T_i)$ ，则设 $R\text{-timestamp}(Q_k) = TS(T_i)$ 。

2. 若事务 T_i 发出 **write(Q)** 指令

- 若 $TS(T_i) < R\text{-timestamp}(Q_k)$ ，则事务 T_i 将被回滚（因为它想写入的版本已经被更晚的事务读取了）。
- 若 $TS(T_i) = W\text{-timestamp}(Q_k)$ ，则 Q_k 的内容将被覆盖（同一事务的修正）。
- 否则（即 $TS(T_i) > R\text{-timestamp}(Q_k)$ ），创建一个新版本 Q_{new} ：
 - 初始化 $W\text{-timestamp}(Q_{\text{new}})$ 和 $R\text{-timestamp}(Q_{\text{new}})$ 为 $TS(T_i)$ 。

观察：

- 读请求总是成功的。
- 若在时间戳序列中存在晚于 T_i 的其他事务读取了旧数据，则 T_i 的写入操作将被拒绝。

多版本两阶段锁定 (Multiversion 2-Phase Locking)

区分 只读事务 与 更新事务。

更新事务 (Update Transactions):

- 获取读写锁，并在事务结束前保持所有锁。即更新事务遵循**严格两阶段锁定 (Rigorous 2PL)**。
- 读取数据项时，可返回该项的最新版本。
- 事务 T_i 对 Q 的首次写入操作将生成数据项 Q 的新版本 Q_{new} 。
 - $W\text{-timestamp}(Q_{new})$ 初始设置为 ∞ (表示未提交)。
- 当更新事务 T_i 完成时，将执行提交处理：
 - 数据库中维护一个全局计数器 **ts-counter** 用于分配时间戳。
 - 读取 **ts-counter** 时也采用锁定机制。
 - 设置 $TS(T_i) = \text{ts-counter} + 1$ 。
 - 将所有由 T_i 创建的新版本 Q_{new} 的时间戳设置为 $TS(T_i)$ 。
 - $\text{ts-counter} = \text{ts-counter} + 1$ 。

只读事务 (Read-only Transactions):

- 在开始执行时会被分配一个时间戳 $TS = \text{ts-counter}$ 。
- 遵循多版本时间戳排序协议进行读取操作。
 - **不获取任何锁。**
 - 读取在 TS 之前（或相等）且具有最大时间戳的版本。
- 即使在只读事务开始后，有其他更新事务提交并增加了 **ts-counter**，只读事务依然能看到一致的旧快照。
- 这种机制保证了**可串行化**的调度。

MVCC 实施问题

- 创建多版本会增加存储开销。
- 需要清理过时的版本 (Garbage Collection)。
- 需要维护版本的时间戳。
- 需要处理事务的提交和回滚。
- 需要处理并发事务的冲突。

快照隔离 (Snapshot Isolation)

快照隔离是一种特殊的多版本并发控制 (MVCC) 机制，允许事务在开始时获取数据库的快照，并在该快照上执行操作，而不会受到其他事务的干扰。

动机:

- 需要读取大量数据的决策支持查询 (Decision Support Queries) 会与更新少量数据的 OLTP 事务产生并发冲突。
- 这种冲突可能导致性能不佳。

解决方案 1: 采用多版本两阶段锁定机制

- 为只读事务提供数据库状态的逻辑快照。
- 只读事务对快照执行读取操作。
- 更新（读写）事务采用常规锁定机制。
- **问题：**系统如何判断某笔交易是否为只读？

解决方案 2 (部分)：向每个用户提供同步数据库状态快照

- 事务对快照执行读取操作。
- 对更新的数据项启用两阶段锁定机制。
- **问题**：可能导致诸如**更新丢失**等各类异常情况。
- **更优方案**：完整的快照隔离级别（Snapshot Isolation Level），通过“先提交者胜”（First-Committer-Wins）规则解决写冲突。